Федеральное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники

Дисциплина: «Основы профессиональной деятельности»

Лабораторная работа №7

«Синтез команд БЭВМ»

Вариант **4381**

Выполнил:
Крамарь Михаил Романович
P3113

Проверила:
Остапенко Ольга Денисовна

г. Санкт-Петербург
2026

Содержание

Текст задания 3

Ход работы 4

    Текст синтезированных микропрограмм...4

    Текст тестовых программ на языке Ассемблера БЭВМ.........................6

    Таблица трассировки циклов исполнения разработанных микрокоманд8

    Методика проверки программы..9

Вывод..10

Текст задания

Синтезировать цикл исполнения для выданных преподавателем команд. Разработать тестовые программы, которые проверяют каждую из синтезированных команд. Загрузить в микропрограммную память БЭВМ циклы исполнения синтезированных команд, загрузить в основную память БЭВМ тестовые программы. Проверить и отладить разработанные тестовые программы и микропрограммы.

1. ***BGC ADDR*** - переход к команде, расположенной по адресу, на которую указывает адресная часть команды, если аккумулятор содержит число, большее чем *8191*
2. Код операции - ***FFXX***
3. Тестовая программа должна начинаться с адреса ***028916***

Ход работы

Текст синтезированных микропрограмм

Подробное описание:

```
                 COMP.   ADDR (E5)       BIT        COMM.   ALU      REG RD
                   ^         ^            ^           ^       ^        ^
E0: 1|000 000|1 | 1110 0101 | 1000 0000 | 0100 | 0000 | 0001 0000
                   ^                   ^               ^                  ^
    Сравниваем с единицей        15-й бит          HTOL               RDAC
```

```
                 COMP.   ADDR (E3)       BIT        COMM.   ALU      REG RD
                   ^         ^            ^           ^       ^        ^
E1: 1|000 000|1 | 1110 0011 | 0010 0000 | 0100 | 0000 | 0001 0000
                   ^                       ^           ^                  ^
    Сравниваем с единицей            13-й бит      HTOL               RDAC
```

```
                 COMP.   ADDR (E5)       BIT        COMM.   ALU      REG RD
                   ^         ^            ^           ^       ^        ^
E2: 1|000 000|0 | 1110 0101 | 0100 0000 | 0100 | 0000 | 0001 0000
                   ^                    ^              ^                  ^
      Сравниваем с нулём           14-й бит         HTOL              RDAC
```

```
     HALT    IO   MEM     REG WR      COMMUTATOR        ALU      REG RD
      ^       ^    ^        ^               ^             ^         ^
E3: 0|0|00 | 00|00 | 0010 0000 | 0000 0001 0001 | 0000 | 0000 0010
                       ^                   ^     ^                    ^
                     WRBR                 SEXT   LTOL                RDCR
```

```
     HALT    IO   MEM     REG WR      COMMUTATOR        ALU      REG RD
      ^       ^    ^        ^               ^             ^         ^
E4: 0|0|00 | 00|00 | 0000 0100 | 0000 0000 1001 | 0000 | 0010 0100
                            ^                 ^    ^           ^    ^
                          WRIP              HTOH  LTOL      RDBR  RDIP
```

```
                 COMP.   ADDR (C4)       BIT        COMM.   ALU      REG RD
                   ^         ^            ^           ^       ^        ^
E5: 1|000 000|0 | 1100 0100 | 0001 0000 | 0001 | 0000 | 0100 0000
```

*В **E5** мы попадаем, если число меньше 8191, сразу переходим к циклу прерывания.*

*Изменения памяти микрокоманд для кастомной команды «**BGC ADDR**»*

Адрес МП	***Микрокоманда***	***Действие ; Комментарии***
E0	*81E5804010*	**If AC(15) = 1 then GOTO E5** ; Переход к E5, если в AC отрицательное число, иначе переход к следующей МК
E1	*81E3204010*	**If AC(13) = 1 then GOTO E3** ; Переход, если 13-й бит старшего байта AC равен единице (в AC 8192 или больше)
E2	*80E5404010*	**If AC(14) = 0 then GOTO E5** ; Сюда попадаем, если 13-й бит старшего байта равен нулю. Переход к E5, если 14-й бит старшего байта AC равен нулю
E3	*0020011002*	**SEXT** ; Расширение знака младшего байта CR (extend sign CR(0…7))
E4	*0004009024*	**BR + IP -> IP** ; Сложение буферного регистра BR с счётчиком команд IP и запись в счётчик IP
E5	*80C4101040*	**GOTO INT @ C4** ; Переход к циклу прерывания (завершающая микрокоманда)

Текст тестовых программ на языке Ассемблера БЭВМ

```
            ORG 0x282
RESULT:     WORD ? ; Ячейка, содержащая 0x1 в случае успешного прохождения
всех тестов, иной результат в противном случае

CHECK1:     WORD 0x0
CHECK2:     WORD 0x0
CHECK3:     WORD 0x0

CONST1:     WORD 16542
CONST2:     WORD -8191
CONST3:     WORD 1335

        ORG 0x289
START:      CALL TEST1 ; Вызов первой тестовой подпрограммы
        CALL TEST2 ; Вызов второй тестовой подпрограммы
        CALL TEST3 ; Вызов третьей тестовой подпрограммы
        LD #0x1 ; Загружаем в AC 0x1 для дальнейшей проверки
        AND CHECK1 ; Побитовое «И» со всеми тестовыми результатами
        AND CHECK2
        AND CHECK3
        ST $RESULT ; Сохраняем в ячейку с меткой RESULT

STOP:       HLT ; Останов.

TEST1:      LD $CONST1 ; Загружаем в AC первое тестовое значение
        WORD 0xFF01 ; BGC ADDR - синтезированная команда (тут мы ожидаем
пропуск прыжка, так как 16542 > 8191)
        JUMP ERROR1 ; Если не сработало, то прыгаем в ошибку
        LD #0x1
        ST CHECK1 ; Сохраняем в CHECK1 0x1

        RET ; Возврат в основную программу

ERROR1:     LD #0x0
        ST $CHECK1 ; В случае неправильного срабатывания записываем 0x0 в
CHECK1
        RET ; Возврат в основную программу

TEST2:      LD $CONST2 ; Загружаем в AC второе тестовое значение
        WORD 0xFF03 ; BGC ADDR - синтезированная команда (тут мы не ожидаем
переход, так как  -8191 < 8191)
        LD #0x1 ; Если не выполнен переход (так и надо), записываем 0x1
        ST $CHECK2 ; Сохраняем в CHECK2 0x1, если условие корректно отработано
        RET ; Возврат в основную программу

ERROR2:    LD #0x0 ; Сюда попадаем, если условие почему-то выполнено -> ошибка
        ST $CHECK2 ; Сохраняем в CHECK2 0x0

        RET ; Возврат в основную программу

TEST3:     LD $CONST3 ; Загружаем в AC третье тестовое значение
        WORD 0xFF03 ; BGC ADDR - синтезированная команда (тут мы не ожидаем переход, так как  1335 < 8191)
        LD #0x1 ; Если не выполнен переход (так и надо), записываем 0x1
        ST $CHECK3 ; Сохраняем в CHECK3 0x1, если условие корректно отработано
        RET ; Возврат в основную программу

ERROR3:    LD #0x0 ; Сюда попадаем, если условие почему-то выполнено -> ошибка
        ST $CHECK3 ; Сохраняем в CHECK3 0x0

        RET ; Возврат в основную программу
```

Таблица трассировки циклов исполнения разработанных микрокоманд

- Пусть в аккумуляторе находится значение 16542_{10}. Выполняется команда ***BGC 0x295***, расположенная по адресу **0293_{16}.**

МР до выборки МК	***Содержимое памяти и регистров процессора после выборки и исполнения микрокоманды***								
	MR	***IP***	***CR***	***AR***	***DR***	***BR***	***AC***	***NZVC***	***MP (СчМК)***
E0	*81E5804010*	294	FF01	293	FF01	293	409E	0000	E1
E1	*81E3204010*	294	FF01	293	FF01	293	409E	0000	E2
E2	*80E5404010*	294	FF01	293	FF01	293	409E	0000	E3
E3	*0020011002*	294	FF01	293	FF01	0001	409E	0000	E4
E4	*0004009024*	295	FF01	293	FF01	0001	409E	0000	E5
E5	*80C4101040*	295	FF01	293	FF01	0001	409E	0000	C4

Методика проверки программы

1. Записать микропрограмму.
2. Загрузить тестовую программу в память базовой ЭВМ.
3. Запустить основную программу с адреса ***0289***$_{16}$.
4. Дождаться останова (HLT).
5. Проверить значение ячейки памяти с меткой RESULT под номером ***0282***$_{16}$, если записанное там значение 0x1 - все тесты выполнены успешно!

Комментарии к методике:

- Для проверки синтезированной мной команды используются три постоянных значения: 16542_{10}, -8191_{10} и 1335_{10}.
- Данные константы показывают правильную работу программы со значениями большими чем 8192_{10} и меньшими 8192_{10}, в том числе и отрицательными.
- Результат каждого теста записывается в соответствующую ему ячейку CHECK, где значение 0x1 означает успешное выполнение. Значение 0x0 - результат неверно синтезированной команды.
- При успешном выполнении всех тестов значение ячейки RESULT станет 0x1, иначе - 0x0.

Тестовое значение		***Теоретический результат***		***Полученный результат***
CONST1	16542_{10}	**CHECK1**	0x1	0x1
CONST2	-8191_{10}	**CHECK2**	0x1	0x1
CONST3	1335_{10}	**CHECK3**	0x1	0x1

Вывод

В ходе выполнения лабораторной работы №7 я изучил алгоритм синтеза собственной команды ассемблера БЭВМ с помощью микропрограмм, более подробно изучил микропрограммное устройство управления, узнал о типах микрокоманд. Также я научился определять за какой вентиль отвечает каждый бит в коде микрокоманды, ознакомился с интерпретатором базовой ЭВМ.